Lehrstuhl für Software and Computational Systems
Henrik Wachowitz

Übungen zur Vorlesung
Softwaretechnik – Wintersemester 2025/2026

Übungsblatt 07

Zu bearbeiten bis Mittwoch, dem 03.12.

1 Littles Gesetz

a) In einem Softwareentwicklungsteam wurden über mehrere Wochen hinweg Daten zur Anzahl der gleichzeitig bearbeiteten Aufgaben (Work in Progress, WIP) und zur durchschnittlichen Durchlaufzeit (Time in Process, TIP) in Tagen gesammelt.

Die folgende Tabelle zeigt die gemessenen Werte:

Woche	WIP	TIP (Tage)
1	3	1,5
2	5	2,0
3	7	3,5
4	10	5,0
5	12	8,0
6	15	12,0

i) Berechnen Sie für jede Woche den Durchsatz (in Aufgaben pro Tag) mithilfe von Little's Gesetz.

ii) Bestimmen Sie, bei welchem WIP-Wert der Durchsatz maximal ist.

iii) Was passiert mit dem Durchsatz, wenn das Team zu viele Aufgaben gleichzeitig bearbeitet? Welche Empfehlung für eine maximale Anzahl von gleichzeitig bearbeiteten Aufgaben würden Sie dem Team geben?

1.1 Lösung

1. Berechnung des Durchsatzes:

Nach Little's Gesetz gilt: Durchsatz $= \frac{\text{WIP}}{\text{TIP}}$

Woche	WIP	TIP (Tage)	Durchsatz (Aufgaben/Tag)
1	3	1,5	$3/1{,}5 = 2{,}0$
2	5	2,0	$5/2{,}0 = 2{,}5$
3	7	3,5	$7/3{,}5 = 2{,}0$
4	10	5,0	$10/5{,}0 = 2{,}0$
5	12	8,0	$12/8{,}0 = 1{,}5$
6	15	12,0	$15/12{,}0 = 1{,}25$

2. Maximaler Durchsatz:

Der Durchsatz ist bei **WIP = 5** mit **2,5 Aufgaben pro Tag** maximal.

3. Empfehlung: Das Team sollte den WIP auf maximal 5 gleichzeitige Aufgaben begrenzen. Dies ist ein Kernelement der Kanban-Methode: *„Stop starting, start finishing!“* – Weniger parallele Arbeit führt zu schnellerer Fertigstellung.

b) Betrachten Sie das kumulative Flussdiagramm (Abbildung 1) eines Softwareentwicklungsteams.

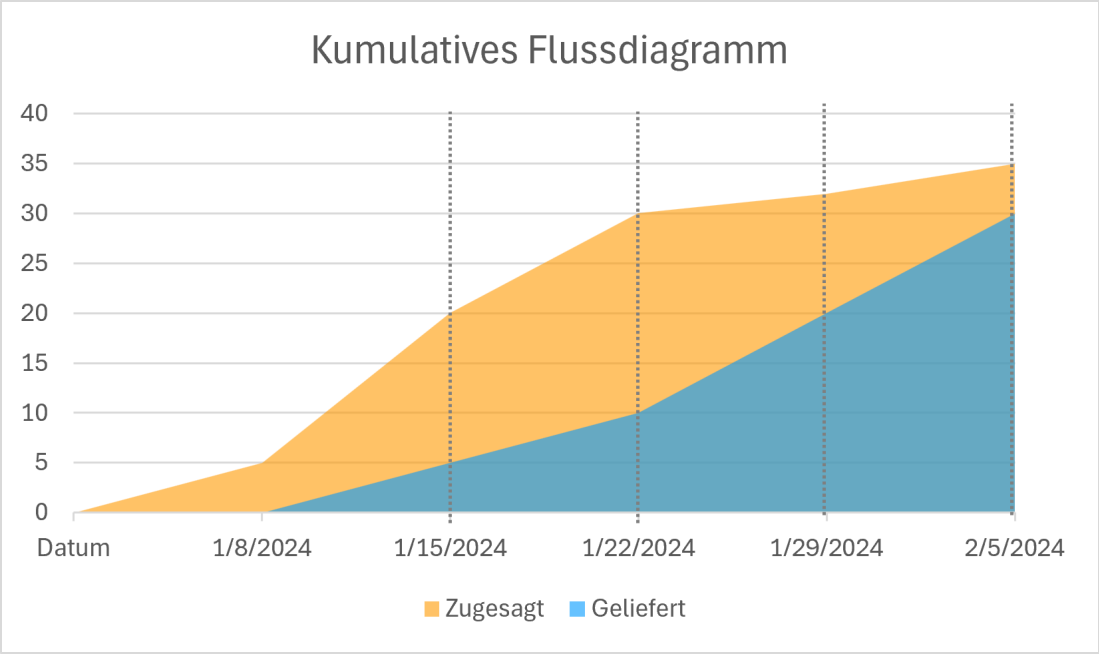

Abbildung 1: Kumulatives Flussdiagramm (CFD) des Teams

a) Berechnen Sie den durchschnittlichen WIP des Teams am 1/15/2024. 15

b) Berechnen Sie die durchschnittliche tägliche Lieferrate vom 1/22/2024 bis zum 2/05/2024. $\frac{10}{7}$ Pakete pro Tag

2 Prinzipien von Kanban

1. Nennen Sie die drei Change Management Prinzipien von Kanban.

   a) Beginne mit dem, was du gerade tust

   b) Vereinbare, dass evolutionäre Veränderung verfolgt wird

   c) Fordere Führung auf allen Ebenen der Organisation

2. Erklären Sie: Wie sollen die Service-Delivery-Prinzipien von Kanban den Fokus der Arbeit einer Organisation verändern? Fokus soll auf Nutzende der Dienstleistung und deren Wert gelegt werden. Dies soll dem Effekt entgegenwirken, dass sich die Organisation auf *sichtbare* Geschäftigkeit der Arbeitenden konzentriert, statt auf den tatsächlichen Wert, der für die Nutzenden geschaffen wird.

3 Kanban Board

1. Skizzieren Sie ein einfaches Kanban Board für ein Softwareentwicklungsteam. Die Stadien des Prozesses sollen *Gewählt*, *In Arbeit*, *Code Review*, *Abnahme* und *Fertig* sein. Fügen Sie WIP-Limits hinzu.
2. Wenn eine Aufgabe in das Stadium *Gewählt* verschoben wird, was bedeutet das für das Team? Es bedeutet, dass das Team sich verpflichtet, die Aufgabe in Angriff zu nehmen und der Kunde sich verpflichtet, die Aufgabe abzunehmen. Es handelt sich hierbei um den **Zusage (Commitment)** Punkt im Kanban Prozess.
3. In manchen Kanban Boards gibt es noch einen Bereich links des *Gewählt* Stadiums, z.B. *Vorschläge* oder *Ideen*. Wie nennt man diesen Bereich? Discovery Kanban
4. Welchen Einfluss hat das tägliche Kanban Meeting auf das Board? Tägliches Standup, bei dem ggf. Anpassungen am Board vorgenommen werden.

4 Fehlerbehandlung II

Betrachten Sie den folgenden Java-Code für einen Service, der Benutzerdaten aus einer Datenbank lädt:

```java
public class UserService {
    private Database db;

    // Version 1: Mit null-Rueckgaben
    public User getUserById(String userId) {
        if (userId == null) {
            return null;  // Ungueltige Eingabe
        }
        try {
            User user = db.findUser(userId);
            return user;  // may be null if userId is not found.
        } catch (DatabaseException e) {
            System.err.println("DB Error: " + e.getMessage());
            return null;  // Fehler beim Zugriff
        }
    }

    // Version 2: Mit Exceptions
    public User getUserByIdV2(String userId)
            throws UserServiceError {
        Objects.requireNonNull(userId, "userId must not be null");
        try {
            User user = db.findUser(userId);
            if (user == null) {
                throw new UserServiceError("User " + userId + " nicht gefunden");
            }
            return user;
        } catch (DatabaseException e) {
            // Datenbank-Fehler weitergeben
            throw new UserServiceError("Datenbankfehler: " + e.getMessage(), e);
```

```
31            }
32        }
33    }
```

a) Betrachten Sie die zwei Versionen der getUserById Methoden.

   i) Welche drei verschiedenen Fehlerzustände können beim Abfragen eines Users entstehen?

      i. Ungültige Eingabe (userId == null) – Programmierfehler

      ii. Benutzer nicht gefunden (user == null) – Erwartbare Situation

      iii. Datenbankfehler (DatabaseException) – Umgebungsfehler

   ii) Wie werden Fehler in den beiden Versionen kommuniziert?

      - Version 1: null-Rückgaben, intransparent → Fehlerquelle unklar
      - Version 2: Exceptions, explizit → klare Fehlerursache

b) In Version 2 des Codes wird ein null-Test zum Methodenbeginn durchgeführt.

   i) Warum ist dieser Test sinnvoll? Verhindert, dass die Methode mit null parametern aufgerufen wird. Macht das ablehnen von null-Parametern explizit.

   ii) Welche Klasse von Fehlern sollen so gefunden werden? Programmierfehler

   iii) Wie nennt sich diese Fehlerbehandlungsstrategie? Fail Fast

c) Nehmen Sie nun an, anstelle eines Fehlers bei einem nicht gefundenen User möchten Sie einen Rückgabewert verwenden um dies zu signalisieren. Welches Konstrukt würden Sie in Java verwenden und warum? Optional<User>, da es explizit signalisiert, dass kein Wert vorhanden sein kann.

d) Schreiben Sie eine Methode `getUserFromDbWithRetry`, die einen String `userId` als Argument erhält. Bei einem Datenbankfehler wird bis zu 3-mal versucht, den User zu laden, bevor ein Fehler zurückgegeben wird. Verwenden Sie dabei die Strategie des exponentiellen Backoffs mit Startwert 100ms. Sie dürfen die Lösung auch in Python schreiben.

```java
1   private User getUserFromDbWithRetry(String userId)
2           throws DatabaseException {
3       final int maxRetries = 3;
4       int retryDelayInMs = 100;
5       for (int attempt = 1; attempt <= maxRetries; attempt++) {
6           try {
7               return db.findUser(userId);
8           } catch (DatabaseException e) {
9               if (attempt == maxRetries) {
10                  throw e; // Letzten Fehler weitergeben
11              }
12              try {
13                  Thread.sleep(retryDelayInMs);
14              } catch (InterruptedException ie) {
15                  Thread.currentThread().interrupt();
16                  throw new DatabaseException("Retry
                       interrupted", ie);
17              }
18              retryDelayInMs *= 2; // Exponentielles Backoff
19          }
```

```
        }
        throw new IllegalStateException("Unreachable code");
    }
```

Python Version:

```python
def get_user_from_db_with_retry(user_id):
    max_retries = 3
    retry_delay = 0.1  # in seconds
    for attempt in range(1, max_retries + 1):
        try:
            return db.find_user(user_id)
        except DatabaseException as e:
            if attempt == max_retries:
                raise e
            time.sleep(retry_delay)
            retry_delay *= 2
```

Allgemeine Hinweise:

- Die vorgestellte Lösung ist lediglich ein Lösungsvorschlag. Es wird kein Anspruch auf Vollständigkeit oder Korrektheit erhoben.
- Das Blatt bereitet Sie auf die Vorlesung am **Mittwoch, dem 03.12.** vor.
- Das Blatt wird in der Übung am **04.12.** besprochen.
- Wir empfehlen Ihnen auch das Quiz in Moodle zu bearbeiten.